

Lecture 20: TCP Part 2 — Congestion Control and the Modern Picture

EC 441 — Introduction to Computer Networking
Boston University, Spring 2026

Thursday, April 9, 2026

Learning Objectives

By the end of this lecture you should be able to:

1. Explain why TCP congestion control is necessary and how it differs from flow control.
2. Describe the three phases of TCP congestion control: slow start, congestion avoidance (AIMD), and fast recovery.
3. Trace the evolution of `cwnd` and `ssthresh` in response to loss events (timeout and triple duplicate ACK).
4. State the TCP throughput formula and explain its dependence on RTT and loss rate.
5. Explain why high-bandwidth-delay-product paths require window scaling and why Reno's AIMD performs poorly on them.
6. Describe CUBIC's key departure from Reno and why it dominates in modern Linux.
7. Explain what ECN is and why it is preferable to loss-based congestion signals.
8. Describe the AIMD fairness argument: why two competing flows converge to equal bandwidth shares.
9. Explain why QUIC was designed and what specific TCP limitations it addresses.

1 The Missing Piece — Congestion Window

Lecture 19 built the full single-connection picture of TCP: reliable delivery, RTT estimation, and flow control. One constraint was stated but not developed:

$$\text{effective window} = \min(\text{cwnd}, \text{rwnd})$$

We defined `rwnd` — the receiver advertises exactly how many bytes of free buffer space it has. We said nothing about `cwnd`. Today we define it.

`cwnd` is the sender's *self-imposed* estimate of how much data the network can currently absorb. It is the heart of TCP congestion control.

Limit	Set by	Protects
<code>rwnd</code>	Receiver	Receiver's buffer from overflow
<code>cwnd</code>	Sender	Network from congestion
Effective window	min of both	Both simultaneously

The distinction between the two controls is fundamental:

- **Flow control is explicit:** the receiver tells the sender exactly how much room it has (via the Window field in every ACK).
- **Congestion control is implicit:** no router ever says “I am congested.” The sender must *infer* network state from packet loss signals and respond conservatively.

TCP uses two distinct loss signals with different severity:

- **Timeout (RTO fires):** severe — the network is likely very congested or partially failed; nothing is getting through.
- **Three duplicate ACKs:** mild — one packet was lost, but later packets are still arriving at the receiver (the dup ACKs prove delivery of segments after the gap).

2 Slow Start

When a TCP connection begins — or after a timeout resets `cwnd` to 1 — the sender has no information about available bandwidth. It must probe the network carefully.

Slow-start rule. Initialize `cwnd` = 1 MSS. For each ACK received, increment `cwnd` by 1 MSS.

Because a window of N MSS segments generates N ACKs, *`cwnd` doubles every RTT*:

RTT	<code>cwnd</code>
0	1 MSS
1	2 MSS
2	4 MSS
3	8 MSS
⋮	⋮

This is *exponential* growth — slow start is not slow in terms of eventual speed, only cautious in its starting point.

ssthresh: the slow-start threshold. Exponential growth would overwhelm the network. TCP maintains a state variable `ssthresh`:

- While `cwnd` < `ssthresh`: **slow start** (exponential growth, +1 MSS per ACK).
- Once `cwnd` ≥ `ssthresh`: **congestion avoidance** (linear growth, +1 MSS per RTT).

`ssthresh` is initialized to a large value (often 65,535 bytes or `rwnd`). The first time congestion is detected, `ssthresh` is set to `cwnd/2`, establishing the boundary for future rounds.

Key Result

Slow start is exponential in `cwnd` growth but *cautious* in its starting point (1 MSS). The name refers to the starting condition, not the growth rate.

3 Congestion Avoidance and AIMD

Once `cwnd` $\geq$ `ssthresh`, TCP switches to *additive increase*:

Congestion-avoidance rule. For each ACK received:

$$\text{cwnd} += \text{MSS} \times \frac{\text{MSS}}{\text{cwnd}}$$

This achieves approximately +1 MSS per RTT regardless of window size.

Multiplicative decrease on loss.

- **Three duplicate ACKs:** `ssthresh` = `cwnd/2`; `cwnd` = `ssthresh` (fast recovery — skip slow start).
- **Timeout:** `ssthresh` = `cwnd/2`; `cwnd` = 1 MSS; restart slow start.

Key Result

AIMD — Additive Increase, Multiplicative Decrease: the mathematical core of TCP congestion control. +1 MSS/RTT when no loss; halve `cwnd` on loss.

Worked cwnd Trace

Starting conditions: `cwnd` = 1 MSS, `ssthresh` = 8 MSS.

Event	cwnd before	cwnd after	ssthresh	Phase
Start	—	1	8	Slow start
ACK (SS)	1	2	8	SS: doubles
ACK (SS)	2	4	8	SS: doubles
ACK (SS, hits thresh)	4	8	8	SS → CA
ACK (CA)	8	9	8	CA: +1/RTT
ACK (CA)	9	10	8	CA
3 dup ACKs	10	5	5	Fast recovery
New ACK, CA resumes	5	6	5	CA
ACK (CA)	6	7	5	CA
ACK (CA)	7	8	5	CA
Timeout	8	1	4	Restart SS
SS resumes	1	2	4	SS
SS resumes	2	4	4	SS → CA
ACK (CA)	4	5	4	CA
⋮				

Note how the timeout at `cwnd` = 8 causes a severe reset (`cwnd` back to 1), while the earlier triple-dup-ACK event at `cwnd` = 10 results in a milder halving to 5.

4 Fast Recovery

Three duplicate ACKs trigger both *fast retransmit* (L19: retransmit the missing segment immediately without waiting for RTO) and *fast recovery*:

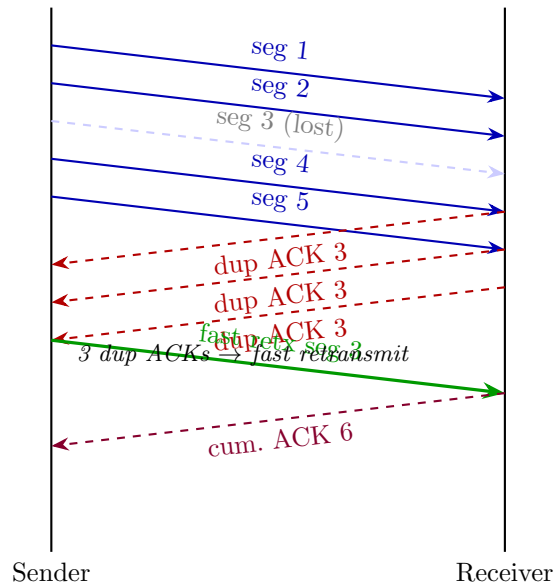
1. Set `ssthresh` = `cwnd` / 2.
2. Set `cwnd` = `ssthresh` (skip slow start — network is not fully broken).
3. Retransmit the missing segment.
4. For each additional duplicate ACK received while in recovery: `cwnd` += 1 MSS. Each dup ACK proves one more segment was delivered to the receiver, so the window can be inflated temporarily.
5. When the first new ACK arrives (acknowledging the retransmitted segment): set `cwnd` = `ssthresh` and return to congestion avoidance.

Contrast with timeout behavior: `cwnd` drops to 1 and full slow start restarts. Timeout is reserved for the scenario where nothing at all is getting through.

Key Result

Fast recovery skips slow start because the duplicate ACKs prove the network is not fully broken — data is still flowing. This avoids an unnecessarily drastic reduction in `cwnd`.

The following ladder diagram illustrates the fast-retransmit trigger. Segments 1–5 are sent; segment 3 is lost; the receiver generates duplicate ACK 3 for every subsequent segment that arrives out of order. After three dup ACKs, the sender immediately retransmits segment 3. The receiver then sends a cumulative ACK 6 acknowledging all data through segment 6.



5 TCP Performance Analysis

Throughput Formula

Under steady-state congestion avoidance with loss rate p , TCP Reno's throughput converges to a well-known result.

Derivation intuition.

- `cwnd` oscillates in a sawtooth between $W/2$ and W ; the time-average is $\approx 3W/4$.
- A loss event occurs when `cwnd` reaches W , having grown from $W/2$. The number of segments sent between loss events is $\approx W^2/2$.
- Therefore the per-segment loss rate satisfies $p \approx 2/(3W^2)$, giving $W \approx \sqrt{2/(3p)}$.
- Throughput = $\frac{\text{avg cwnd} \times \text{MSS}}{\text{RTT}} \approx \frac{(3W/4) \cdot \text{MSS}}{\text{RTT}}$.

Substituting and absorbing constants:

$$\boxed{\text{BW} \approx \frac{\text{MSS}}{\text{RTT}} \cdot \frac{1}{\sqrt{p}}} \quad (1)$$

Key Result

TCP throughput scales as $1/\text{RTT}$ and $1/\sqrt{p}$. Long-RTT paths and high-loss paths both suffer. This formula is the fundamental limit of TCP Reno under steady-state congestion avoidance.

Throughput Calculations

(a) How much throughput can Reno achieve?

MSS = 1460 bytes, RTT = 100 ms, $p = 0.01\%$:

$$\text{BW} \approx \frac{1460 \times 8}{0.1} \times \frac{1}{\sqrt{0.0001}} = \frac{11,680 \text{ b}}{0.1 \text{ s}} \times 100 \approx 11.68 \text{ Mb/s}$$

(b) What loss rate is needed for 10 Gb/s?

With RTT = 100 ms and target BW = 10 Gb/s:

$$p < \left(\frac{\text{MSS}}{\text{RTT} \times \text{BW}} \right)^2 = \left(\frac{1460 \times 8}{0.1 \times 10 \times 10^9} \right)^2 \approx 1.4 \times 10^{-10}$$

Roughly *one loss per 7 billion segments*. This is why long-fat networks (high BDP) are a fundamental challenge for TCP Reno.

6 Bandwidth-Delay Product and Window Scaling

The *bandwidth-delay product* (BDP) is the number of bytes that fill the pipe end to end:

$$\text{BDP} = \text{BW} \times \text{RTT}$$

For the sender to keep the pipe full, `cwnd` must be at least BDP. The standard TCP Window Size field is 16 bits, allowing a maximum of 65,535 bytes ($\approx 64 \text{ KB}$).

Problem. $1 \text{ Gb/s} \times 100 \text{ ms} = 12.5 \text{ MB}$ — nearly 200 times the 64 KB limit. Without options, the maximum achievable throughput on this path is:

$$\frac{64 \text{ KB}}{0.1 \text{ s}} = 5.2 \text{ Mb/s} \quad (\text{less than } 1\% \text{ of link capacity})$$

Window Scale option (RFC 7323). A TCP option negotiated at SYN/SYN-ACK time. Each endpoint announces a *shift count* $n \in [0, 14]$; the effective window = advertised value $\times 2^n$. Maximum effective window: $65,535 \times 2^{14} \approx 1 \text{ GB}$.

BW	RTT	BDP	Min wscale needed
100 Mb/s	10 ms	125 KB	1 ($\times 2$)
1 Gb/s	100 ms	12.5 MB	8 ($\times 256$)
10 Gb/s	100 ms	125 MB	12 ($\times 4096$)

The Window Scale option is negotiated *only* at SYN/SYN-ACK. If either endpoint omits it, the session falls back to 64 KB and high-BDP performance is capped.

7 TCP Reno vs. CUBIC

TCP Reno's Limitations on High-BDP Paths

TCP Reno's additive increase is *RTT-clocked*: one AIMD step per RTT. On a high-BDP path (say, RTT = 100 ms), recovery from a single loss event requires many RTTs to climb `cwnd` back to W . The sawtooth has a long period and a low fill factor.

RTT unfairness. A flow with RTT = 10 ms gets 10 AIMD steps per second while a flow with RTT = 100 ms gets only 1. In a shared bottleneck, the shorter-RTT flow acquires disproportionately more bandwidth. This is *RTT unfairness*, a structural property of Reno's per-RTT update rule.

CUBIC (RFC 9438)

CUBIC has been the default congestion-control algorithm in Linux since kernel 2.6.19.

Key idea. Instead of advancing `cwnd` by one step per RTT, CUBIC grows `cwnd` as a cubic function of *wall-clock time* t elapsed since the last loss event:

$$W(t) = C(t - K)^3 + W_{\max}$$

where $W_{\max}$ is the window at the time of the last loss, and K is chosen so that $W(0) = W_{\max}/2$ (the post-loss window) and $W(K) = W_{\max}$ (the window is recovered).

- **Far from $W_{\max}$** ($|t - K|$ large): the cubic term dominates, growth is aggressive.
- **Near $W_{\max}$** ($t \approx K$): growth slows — CUBIC probes cautiously near the estimated bottleneck.
- Because growth is a function of wall-clock time (not RTT count), two flows with different RTTs but the same bottleneck link get equal AIMD steps per second — RTT fairness is restored.

Key Result

CUBIC's cubic curve provides fast recovery far from the previous loss point and caution near it — opposite of Reno's constant linear slope. This makes it efficient on high-BDP links and fairer across flows with different RTTs.

	Growth clock	High-BDP behavior	RTT fairness	Linux default?
TCP Reno	Per RTT (linear)	Slow recovery	Poor	No
CUBIC	Wall clock (cubic)	Fast recovery	Good	Yes (since 2.6.19)

8 Explicit Congestion Notification (ECN)

Problem with loss-based signals. When a router's queue overflows, it drops a packet. By the time the sender detects the drop (via timeout or dup ACKs), the queue has *already* overflowed. The dropped packet's data is permanently lost — goodput suffers. The signal is both late and destructive.

ECN mechanism (RFC 3168). A router can signal impending congestion *before* dropping packets:

1. As the router's queue fills (but before overflow), the router sets the **CE (Congestion Experienced)** bit in the IP header of a passing packet.
2. The receiver reflects the CE signal to the sender via the **ECE (ECN-Echo)** flag in a TCP ACK.
3. The sender reduces `cwnd` exactly as if a loss had occurred — but the packet was *delivered*.

Benefits.

- No goodput loss from the dropped packet.
- Earlier signal: queue is building, not yet overflowing.
- Smoother behavior: fewer sawtooth oscillations.

Deployment. Both endpoints *and* every router on the path must support ECN. ECN is widely deployed in datacenters (where all equipment is operator-controlled). On the public Internet, partial deployment means some paths benefit and some do not.

Key Result

ECN separates the congestion signal from packet loss. This is strictly better — the same information arrives earlier and without discarding useful data.

9 AIMD Fairness

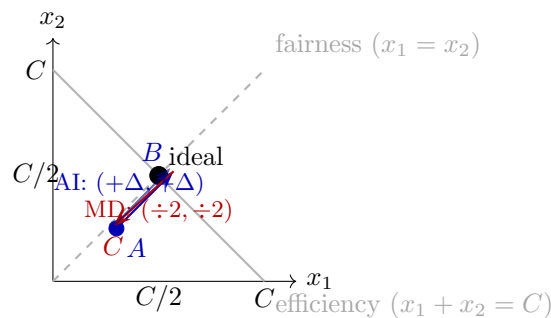
Consider two TCP flows sharing a single bottleneck link with capacity C . Let x_1 and x_2 be their throughputs (in units of bandwidth, with the bottleneck fully shared).

- **Efficiency line:** $x_1 + x_2 = C$ (the pipe is full).
- **Fairness line:** $x_1 = x_2$ (equal shares).
- **Ideal point:** intersection, $(C/2, C/2)$.

Additive increase. Both flows add the same Δ per RTT. The operating point moves in direction $(1,1)$ — parallel to the fairness line. This moves the point toward the efficiency line without changing the fairness gap.

Multiplicative decrease. Both flows halve on congestion: $(x_1, x_2) \rightarrow (x_1/2, x_2/2)$. This is a vector toward the origin. Crucially, the ratio x_1/x_2 is preserved, but the distance from the fairness line shrinks *relative to* the distance from the efficiency line.

Combined effect. AI steps move along $(1,1)$ toward the efficiency line. MD steps move toward the origin, reducing the fairness gap. The trajectory spirals toward the ideal point $(C/2, C/2)$.



Each AI step (blue arrow) moves the operating point toward the efficiency line along direction $(1,1)$. Each MD step (red arrow) moves it toward the origin, reducing the fairness gap. The trajectory converges to the ideal point.

Key Result

AIMD is the unique simple linear update policy that converges to *both* efficiency and fairness simultaneously.

Caveats.

- The proof assumes equal RTTs. TCP Reno penalizes longer-RTT flows (fixed by CUBIC).
- UDP flows do not participate in AIMD and can crowd out TCP flows at a shared bottleneck.
- A single host running N TCP connections receives a share proportional to N , giving it an advantage over a host running a single connection.

10 QUIC

TCP has served the Internet for decades, but several structural limitations have become painful as applications demand lower latency and higher mobility.

Four TCP Limitations that Motivated QUIC

1. **Head-of-line (HOL) blocking.** HTTP/2 multiplexes many logical streams over a single TCP connection. TCP sees one byte-stream; it cannot deliver any data from the stream above a gap. One dropped packet stalls all streams until the retransmitted segment arrives. QUIC delivers each stream independently — loss in one stream does not block others.

2. **Connection setup latency.** TCP’s 3-way handshake requires 1 RTT before any data can be sent. TLS negotiation adds another 1–2 RTTs. Total: 2–3 RTTs before the first byte of application data flows. QUIC combines the transport and TLS 1.3 handshakes: 1 RTT for new connections; 0 RTT for resumed connections (with caveats around replay-attack resistance).
3. **Ossification.** Every middlebox (firewall, NAT, load balancer, DPI device) on the Internet interprets TCP headers and may modify or block unfamiliar options. Deploying new TCP extensions is practically impossible. QUIC runs over UDP; its entire header (beyond the connection ID) is encrypted. Middleboxes cannot inspect or modify the QUIC payload.
4. **Connection migration.** A TCP connection is identified by its 4-tuple (src IP, src port, dst IP, dst port). Changing networks (WiFi → cellular) changes the source IP/port, breaking the connection. QUIC identifies connections by a *connection ID* that is independent of network addresses. Connections survive address changes seamlessly.

What QUIC Is

- A transport protocol built on top of UDP.
- Mandatory TLS 1.3 encryption (no unencrypted QUIC).
- Multiple independent streams per connection (no HOL blocking).
- Its own congestion control (commonly CUBIC or BBR), implemented in user space.
- HTTP/3 = HTTP/2 semantics over QUIC. Used by YouTube, Google Search, and roughly 25% of Internet traffic by volume.

Key Result

QUIC does not replace TCP for all uses — it targets web traffic where setup latency, HOL blocking, and mobility matter. TCP remains dominant for bulk transfers, long-lived connections, and the vast majority of non-web traffic.

UDP / TCP / QUIC Comparison

The three transport protocols covered in this unit occupy different points in the design space. Table 1 summarizes their key properties.

The table highlights the design progression: UDP provides the minimum possible transport-layer service (port-based demultiplexing only); TCP adds reliability, ordering, flow control, and congestion control at the cost of connection overhead and HOL blocking; QUIC reimplements TCP’s reliability features on top of UDP, adds mandatory encryption, and eliminates HOL blocking across streams.

11 The Socket Programmer’s View

All of TCP’s machinery — RTT estimation, congestion control, flow control, retransmission, window management — is *invisible* to the application. The application calls `connect()`, `send()`, `recv()`, and `close()`; the OS kernel handles everything else.

Feature	UDP	TCP	QUIC
Connection setup	None	3-way handshake (1 RTT)	1 RTT (0 RTT resumed)
Reliability	None	Yes (cumul. ACK + retx)	Yes (per stream)
Ordering	None	Yes (byte stream)	Yes (per stream)
Flow control	None	rwnd	Yes
Congestion control	None	AIMD (cwnd)	CUBIC / BBR
HOL blocking	N/A	Yes (one byte stream)	No (indep. streams)
Encryption	None (app layer)	TLS separate	TLS 1.3 built-in
Header size	8 bytes	20+ bytes	Variable (encrypted)
Runs over	IP	IP	UDP
Typical use cases	DNS, streaming, games	Web, SSH, bulk transfer	HTTP/3, YouTube

Table 1: Comparison of UDP, TCP, and QUIC.

Relevant socket options.

- **TCP_NODELAY**: disables Nagle’s algorithm. With Nagle enabled, the OS buffers small writes until a full MSS or the previous data is ACKed. **TCP_NODELAY** forces immediate transmission — important for interactive or low-latency applications.
- **TCP_INFO** (Linux only): exposes the kernel’s internal TCP state as a `struct tcp_info`. Readable by `getsockopt()` to obtain `cwnd`, `ssthresh`, `rtt`, retransmit counts, and more. Invaluable for performance debugging.
- **SO_RCVBUF** / **SO_SNDBUF**: control the socket receive and send buffer sizes. The effective window is limited by these buffers — on high-BDP paths, the default buffers (typically 87 KB on Linux) cap throughput. Set to at least $\text{BDP} \times 2$ for full utilization.

ss -ti: Observing Kernel TCP State

The `ss` (socket statistics) tool with `-ti` flags shows the TCP internal state for each active connection directly from the kernel:

```
$ ss -ti
tcp ESTAB 0 0 192.168.1.5:45678 93.184.216.34:443
   cubic wscale:7,7 rto:204 rtt:4.5/1.1 ato:40 mss:1448
   rcvmss:1448 advmss:1448 cwnd:10 ssthresh:10
   bytes_sent:45678 bytes_retrans:0 bytes_acked:45678
   segs_out:32 segs_in:20 data_segs_out:31
   send 257.6Mbps lastsnd:8 lastrcv:8 lastack:8
   pacing_rate 309.1Mbps delivery_rate 257.6Mbps
   delivered:31 app_limited:1 rcv_rtt:4.5
   rcv_space:14600 rcv_ssthresh:64076 minrtt:3.2
```

Key fields:

- **cubic**: congestion-control algorithm (Linux default since kernel 2.6.19).
- **rtt:4.5/1.1**: SRTT = 4.5 ms, RTTVAR = 1.1 ms — directly the Jacobson estimator from L19.
- **rto:204**: RTO = SRTT + 4 × RTTVAR $\approx 4.5 + 4.4 = 8.9$ ms, but RFC 6298 floor is 200 ms — floor visible here.
- **cwnd:10**: current congestion window (in segments).
- **ssthresh:10**: **ssthresh** equal to **cwnd** — sitting at the CA/SS boundary.
- **app_limited:1**: the application is not producing data fast enough to fill **cwnd** — the bottleneck is the app, not the network.

Discussion Questions

1. Why does TCP Reno perform poorly on satellite links (high RTT, high BDP) even at very low loss rates?
2. Two TCP flows share a 100 Mb/s bottleneck: one with RTT = 10 ms, the other with RTT = 100 ms. In steady state, do they each receive 50 Mb/s? Why or why not? How does CUBIC change the answer?
3. If a router uses ECN instead of dropping packets, what happens to the TCP throughput formula? Does the formula still apply?
4. A UDP-based video application has no rate control. What happens to co-existing TCP flows? Is this fair?
5. HTTP/2 over TCP claims to solve head-of-line blocking with stream multiplexing. Why does QUIC claim HTTP/2 still has HOL blocking?
6. Why must QUIC implement its own congestion control rather than relying on the OS TCP stack?

References

1. Wikipedia: TCP congestion control. https://en.wikipedia.org/wiki/TCP_congestion_control
2. Wikipedia: CUBIC TCP. https://en.wikipedia.org/wiki/CUBIC_TCP
3. Wikipedia: Explicit Congestion Notification. https://en.wikipedia.org/wiki/Explicit_Congestion_Notification
4. Wikipedia: QUIC. <https://en.wikipedia.org/wiki/QUIC>
5. Wikipedia: BBR (network congestion control). [https://en.wikipedia.org/wiki/BBR_\(network_congestion_control\)](https://en.wikipedia.org/wiki/BBR_(network_congestion_control))
6. Wikipedia: Bandwidth-delay product. https://en.wikipedia.org/wiki/Bandwidth-delay_product
7. RFC 5681: TCP Congestion Control. <https://www.rfc-editor.org/rfc/rfc5681>
8. RFC 9438: CUBIC for Fast and Long-Distance Networks. <https://www.rfc-editor.org/rfc/rfc9438>
9. RFC 3168: The Addition of Explicit Congestion Notification (ECN) to IP. <https://www.rfc-editor.org/rfc/rfc3168>